RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES · BASAR

Basara Attendance Build Brief

One application, built by the whole class. You will replace the paper attendance sheet that gets retyped into Excel every night — with a system that verifies you were in the room, that it was really you, and that answers questions in plain English.

THE CLASS
106 students · 19 groups

TRACKS
BE · FE · UI/UX · ML · SPD

SPRINT 1 ISSUED
31 July 2026

What We Are Building, and Why

Start with the problem. It is in this room.

THE PROBLEM

A paper sheet, and someone retyping it at night.

Attendance for this course is taken on paper. Student 1 to student n, filled in by hand, passed along the row. Someone on the team then sits in the evening and types all of it into an Excel sheet. It is slow, it is error-prone, and a sheet passed down a row is trivially easy to fill in for a friend who is not here.

WHAT WE BUILD INSTEAD

The class ships the tool that fixes the class.

You mark your own attendance from your own phone, from inside this room. The retyping disappears. Every one of you will have built the system that made it disappear — and that is a far better answer in an interview than any assignment.

The end state, in one sentence

A student opens a URL on their phone, marks attendance from inside the classroom, is **verified by their own face**, and staff ask the data a question **in plain English** and get a true answer back.

Three rules everything else hangs off

#	RULE	WHY IT IS NON-NEGOTIABLE
1	**One mark per student per session**	Enforced by a database constraint, not by an `if` statement. A check followed by a write is a race condition, and two taps on a slow connection will find it.
2	**The client is never trusted**	Anything a phone can set, a phone can lie about — coordinates, timestamps, the photo itself. The server decides. Every time.
3	**Nothing leaves this server**	Attendance photos and face embeddings stay on the class machine. Not S3, not a commercial cloud, not outside India. This one is a promise to your classmates.

What each of you walks away holding

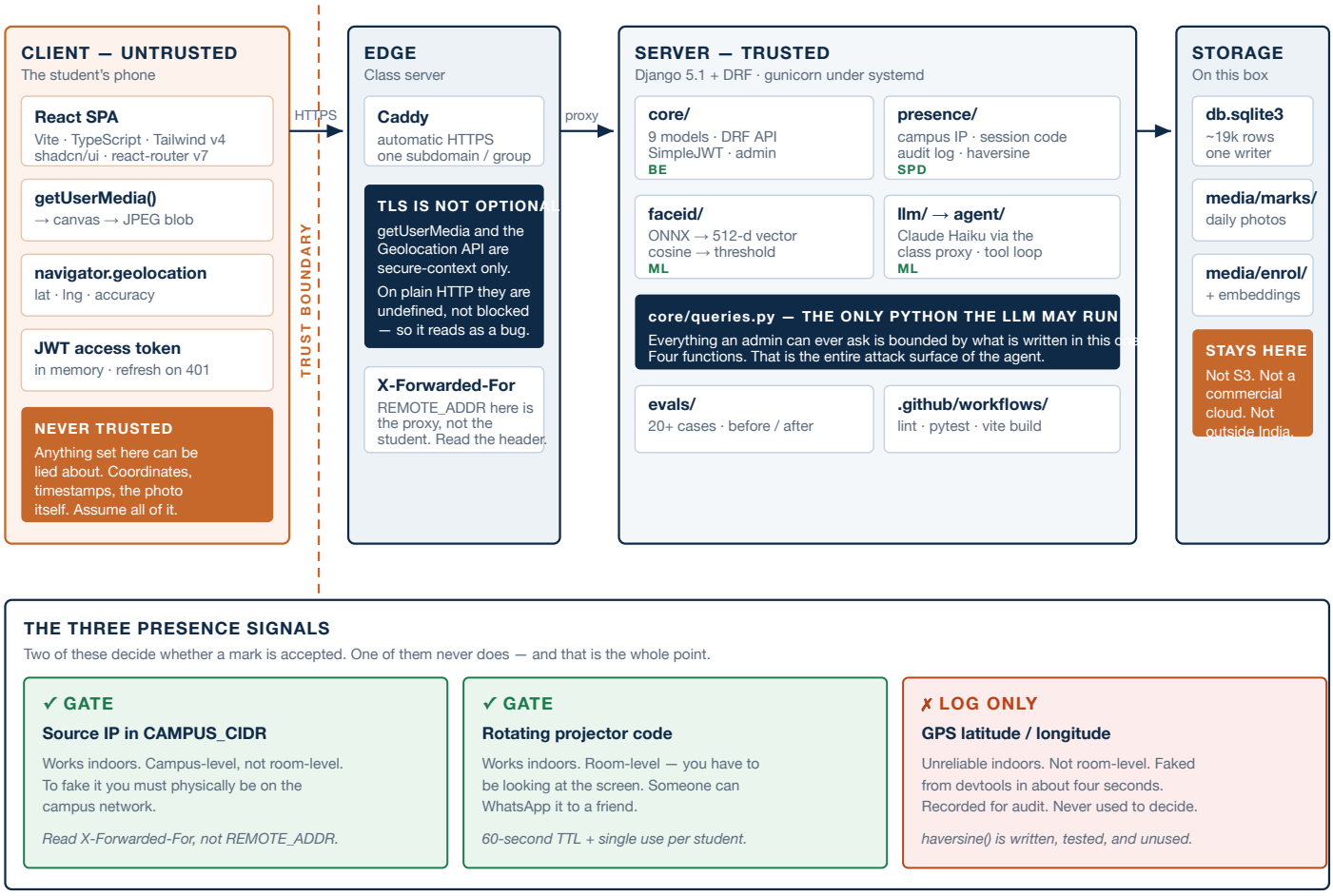
- A **live HTTPS URL** anyone can open on a phone
- A **branch and merged pull requests** with your name on them
- A **face-match threshold you chose** from measured data, and can defend
- A **presence check** you can explain *and* attack
- An **LLM feature** whose cost per call you can state in rupees
- Real experience of **code review** across nineteen teams

A note on the reference implementation

You have been given a **working production attendance system** to port from — the same stack, already deployed and in real use. It is genuinely useful, and it is also genuinely flawed: plaintext passwords, a login token that is generated and then never checked again, and every endpoint open to the world. Its own authors documented all of this. Most of you have never read production code before. You are about to read some, find the holes yourselves, and fix them. **Port from it. Do not copy it.**

Architecture

Where your track sits, and where the trust boundary falls.



THE THREE PRESENCE SIGNALS

Two of these decide whether a mark is accepted. One of them never does — and that is the whole point.

✓ GATE

Source IP in CAMPUS_CIDR

Works indoors. Campus-level, not room-level. To fake it you must physically be on the campus network.

Read X-Forwarded-For, not REMOTE_ADDR.

✓ GATE

Rotating projector code

Works indoors. Room-level — you have to be looking at the screen. Someone can WhatsApp it to a friend.

60-second TTL + single use per student.

✗ LOG ONLY

GPS latitude / longitude

Unreliable indoors. Not room-level. Faked from devtools in about four seconds. Recorded for audit. Never used to decide.

haversine() is written, tested, and unused.

Every box is owned by exactly one track, and every arrow is a contract between two groups. If you change what crosses an arrow, the group on the other side finds out from your pull request — not from a broken build.

Stack, Repo and Process

What we are using, why, and how the code moves.

The stack

LAYER	CHOICE	WHY THIS ONE
Frontend	Vite · React 18 · TypeScript · Tailwind v4 · shadcn/ui · react-router v7	The reference app is already on exactly this stack, so this is a port rather than a build from nothing.
Backend	Django 5.1 · Django REST Framework	Batteries included, and the nine models you need are already written.
Auth	JWT via `djangorestframework-simplejwt`	Student ID and password. No second factor — the complexity budget is spent on faces and presence instead.
Database	**SQLite**	106 students × 5 subjects × ~36 sessions ≈ 19,000 rows. One writer, one machine. Postgres would add an install step per team and buy us nothing. **This is a decision, not an oversight.**
Face matching	ONNX Runtime · MobileFaceNet/ArcFace · 512-d	Runs locally, and — unlike a hosted vision API — returns a **similarity score**, which is what makes a threshold possible.
LLM	Claude Haiku via the class proxy	One shared key, spend visible to the instructor, no card needed by anyone.
Serving	Caddy · gunicorn · systemd	Automatic HTTPS with no certificate wrangling — and without HTTPS, the camera does not work at all.

Branches

```
main          protected  deploys to production
 └─ staging                release candidates
     └─ dev                integration — every pull request targets this
         └─ team/<your-group>      e.g. team/femmeforce, team/grp5
```

Pull requests go `team/*` → `dev`, reviewed by **another group on your track**. **All nineteen branches already exist** — check yours out, do not create it; the names are on the last page.

Today, before you leave the room

1. Clone the repository, then `git checkout team/<your-group-slug>`.
2. Find your track page in this brief. Read the architecture diagram opposite it.
3. Backend and ML: `cd backend`, then `python3.12 -m venv .venv && pip install -r requirements.txt`. **Python 3.12 or 3.13 — not 3.14**, which has no prebuilt wheels for numpy or onnxruntime.
   Frontend and UI/UX: `cd frontend`, then `npm install && npm run dev`.
4. Make **one real commit** — the smallest item on your backlog. Not a README edit.
5. `git push origin team/<your-group-slug>` and open a **draft pull request** into `dev`. No push access yet? Fork it — the README shows both routes.

The backend runs, but it answers exactly one route

`backend/core/urls.py` ships as a **stub** — enough for `runserver` to start and `/api/health/` to return `{"ok":true}`, and nothing more. **GRP12 writes the real URLconf**, and the other seven endpoints are listed in a comment at the top of that file. Nobody is blocked while they do it, but nothing else works until they have. If you are waiting, read `backend/core/models.py` — nine models, already finished, and every track reads from them.

Where things live

Three directories at the root. `backend/` is Django and DRF — **`manage.py` and `requirements.txt` are in there, not at the top**. `frontend/` is React and Tailwind. `docs/` is this brief, the team list and the decision log.

Repository: `github.com/Solve-Systems-Lab/basara-attendance` — default branch `dev`.

How One Attendance Mark Travels

Twelve steps, seven groups, six handovers. This is the whole application in one table.

#	WHERE	WHAT HAPPENS	WHO BUILDS IT
1	THE PHONE	**Capture the photo** getUserMedia → canvas → JPEG File	**DRAGONS** FE
2	THE PHONE	**Read the position** getCurrentPosition, high then low accuracy	**DRAGONS** FE
3	THE PHONE	**Type the projector code** 6 characters, visible only in the room	**DRAGONS** FE
	↓ handover — **DRAGONS** to **GRP8**		
4	THE PHONE	**POST multipart** + Authorization: Bearer <access>	**GRP8** FE
	↓ handover — **GRP8** to **MINIBYTES**		
5	EDGE	**Terminate TLS** set X-Forwarded-For to the real client	**MINIBYTES** SPD
	↓ handover — **MINIBYTES** to **GRP5**		
6	DJANGO	**Resolve the token** JWT → request.user, never a user_id field	**GRP5** BE
	↓ handover — **GRP5** to **BHUVANESHWARI**		
7	CHECKS	**Campus IP?** check_campus_ip(XFF) — GATE	**BHUVANESHWARI** SPD
8	CHECKS	**Live code?** verify_session_code() 60s, single use — GATE	**BHUVANESHWARI** SPD
	↓ handover — **BHUVANESHWARI** to **BALAJI**		
9	CHECKS	**Is it them?** faceid.verify() → cosine score	**BALAJI** ML
	↓ handover — **BALAJI** to **GRP7**		
10	DJANGO	**Apply the rules** threshold, one-per-day, Sunday, holiday	**GRP7** BE
	↓ handover — **GRP7** to **GRP7 / GRP12**		
11	DATABASE	**Write it once** AttendanceRecord + AuditLog, atomic	**GRP7 / GRP12** BE
	↓ handover — **GRP7 / GRP12** to **DRAGONS**		
12	THE PHONE	**Show the result** 201 confirmed, or a readable 403 / 409	**DRAGONS** FE

Read your own step, then the one directly above it

The step above yours decides what arrives at your door; the step below depends on what you send. Those are the teams to talk to — not all nineteen others. What crosses each handover is fixed on the next page, so you do not have to negotiate it in the corridor.

*Steps 7 and 8 are **gates** — fail either and the mark is refused. Step 9 produces a score, not a verdict; BALAJI's threshold decides what it means. The coordinates from step 2 are stored and **never** used to accept or reject.*

The API Contract — Identity

Frozen on day one. Backend implements it, frontend mocks it — neither waits for the other.

This is the most important page in this document

Eight groups — four backend, four frontend — build against these shapes without talking to each other. **If you need to change one, that is a pull request against this document first**, and the group on the other side reviews it. The app you are porting from has five endpoints the frontend calls that the backend never implemented, and twelve the backend serves that nothing calls. That is what happens without this page.

ENDPOINT	REQUEST	RESPONSE
POST `/api/auth/login/` GRP5	`{"roll_no": "B201463", "password": "..."}`	`200 {"access": "", "refresh": "", "user": {"id": 1, "roll_no": "B201463", "name": "...", "is_staff": false}}` `401 {"detail": "Invalid roll number or password"}`
POST `/api/auth/refresh/` GRP5	`{"refresh": ""}`	`200 {"access": ""}` `401 {"detail": "Token is invalid or expired"}`
GET `/api/auth/me/` GRP5	`Authorization: Bearer`	`200 {"id": 1, "roll_no": "B201463", "name": "...", "is_staff": false}` `401 {"detail": "Authentication credentials were not provided."}`

*`login` and `refresh` are the only endpoints in the whole API that do not need a token. Everything else returns **401** without `Authorization: Bearer <access>`, and takes the student **from the token** — never from a field the client sent. `GET /api/health/` is already built and returns `{"ok": true}`.*

The API Contract — Attendance and Admin

The four endpoints the app is actually for. Same rule: change it by pull request, not in the corridor.

ENDPOINT	REQUEST	RESPONSE

POST `/api/attendance/mark/` GRP7	`multipart/form-data photo= latitude=18.123456 longitude=78.123456 accuracy_m=12.5 session_code=7K2M9Q`	`201 {"id": 42, "status": "PRESENT", "marked_at": "2026-08-01T17:05:11+05:30", "face_score": 0.71}`
		`400 bad payload · 401 no token`
		`403 {"detail": "...", "reason": "off_campus|bad_code|face"}`
		`409 {"detail": "Already marked today"}`

GET `/api/attendance/today/` CLAUDE	`Authorization: Bearer`	`200 {"has_marked": true, "record": {"id": 42, "status": "PRESENT", "marked_at": "...", "photo_url": "..."}}` `200 {"has_marked": false, "record": null}`
GET `/api/attendance/my-records/` CLAUDE	`?year=2026&month=8`	`200 {"records": [{"date": "2026-08-01", "status": "PRESENT", "marked_at": "..."}], "summary": {"present": 12, "absent": 2, "percentage": 85.7}}` `400 {"detail": "year and month are required"}`
GET `/api/admin/overview/` CLAUDE	`?date=2026-08-01 (staff only)`	`200 {"date": "2026-08-01", "total": 82, "present": 70, "absent": 12, "rate": 85.4, "students": [{"roll_no": "...", "name": "...", "status": "PRESENT"}]}` `403 {"detail": "Staff only"}`

Two details that decide whether this can be cheated

The mark endpoint takes **no** `user_id` — the student comes from the token, because anything the client can name, an attacker can name too. And `marked_at` is stamped by the **server**, not sent by the phone; a client-supplied timestamp lets anyone backfill last Tuesday. Latitude and longitude *are* accepted and stored, and are **never** used to accept or reject.

Order of Work

Six things gate everything else. If you own one of them, the class is waiting on you.

GROUP	SHIP THIS FIRST	WHO IS STOPPED UNTIL YOU DO	BY
NEURALTEAM	**core/urls.py + settings**	the other 19 groups	**Day 1**
NEXAAI	**the five status colour names**	6 groups	**Day 1**
GRP5	**login / refresh / me**	all 4 FE groups	**Day 2**
MINIBYTES	**HTTPS**	DRAGONS — no TLS, no camera	**Day 2**
MINIONS	**design tokens**	3 UI/UX + 4 FE groups	**Day 2**
ANUSHA	**embed() + cosine()**	BALAJI, then GRP7	**Day 4**

Blocked? Do this instead of waiting.

Frontend — build against the contract with mock data and swap the base URL later. You should never be idle waiting for an endpoint to exist.
Backend — write the serializer and its tests before the view exists.
ML — you run on seeded data and never need the frontend at all.
UI/UX — build components in isolation with fake props.

Stuck for more than fifteen minutes is a question for the room, not a reason to stop. The group you are waiting on is named on the next page — go and talk to *them*, not to everyone.

The one rule that keeps twenty branches from colliding

Edit files inside **your own directory**. If you need a change in someone else's, open a pull request against `dev` and tag them — do not edit it yourself and do not copy it into yours. The directory-ownership table earlier in this brief says who owns what.

Who Blocks Whom

Find your group. If the middle column says “start now”, there is no excuse to be idle.

GROUP	WAITS ON	IS WAITED ON BY
GRP5 BE	NEURALTEAM	GRP8 GRP7 CLAUDE
GRP7 BE	NEURALTEAM GRP5 BHUVANESHWARI BALAJI	DRAGONS
CLAUDE BE	NEURALTEAM GRP5	GRP14 ASTERIX
NEURALTEAM BE	nothing — start now	GRP5 GRP7 CLAUDE
DRAGONS FE	GRP8 GRP7 MINIONS AGENTS MINIBYTES	—
ASTERIX FE	GRP8 CLAUDE GRP4	—
GRP8 FE	GRP5	DRAGONS ASTERIX GRP14
GRP14 FE	GRP8 GRP5 CLAUDE GRP4	—
GRP4 UI/UX	NEXAAI MINIONS	GRP14 ASTERIX
MINIONS UI/UX	NEXAAI	GRP4 AGENTS DRAGONS GRP14 ASTERIX
AGENTS UI/UX	NEXAAI MINIONS	DRAGONS ASTERIX GRP14
NEXAAI UI/UX	nothing — start now	GRP4 AGENTS MINIONS GRP14 ASTERIX
GRP11 ML	FEMMEFORCE	—
ANUSHA ML	nothing — start now	BALAJI
BALAJI ML	ANUSHA	GRP7
FEMMEFORCE ML	CLAUDE	GRP11
BHUVANESHWARI SPD	nothing — start now	GRP7
ROCKSTARS SPD	nothing — start now	—
MINIBYTES SPD	nothing — start now	DRAGONS
GRP12 SPD	GRP7	—

Red means you need something from them. Green means they need something from you — and that a slip on your side costs more than your own group's time.

Who Is Building What

Nineteen groups · 106 students · five tracks.

TRACK	OWNS	GROUPS	GROUPS	STUDENTS
BE	Backend	GRP5 (6) · GRP7 (5) · CLAUDE (4) · **NEURALTEAM** (3)	**4**	**18**
FE	Frontend	**DRAGONS** (5) · **ASTERIX** (5) · GRP8 (5) · **GRP14** (4)	**4**	**19**
UI/UX	UI / UX	**GRP4** (5) · MINIONS (4) · AGENTS (4) · **NEXAAI** (2)	**4**	**15**
ML	ML & AI	GRP11 (5) · ANUSHA (4) · BALAJI (4) · FEMMEFORCE (3)	**4**	**16**
SPD	Platform	BHUVANESHWARI (5) · ROCKSTARS (3) · MINIBYTES (3) · **GRP12** (3)	**4**	**14**

How these were assigned

Your track starts from the category your group chose. Then it was rebalanced, because self-selection produced **43 students on ML and 5 on frontend** — forty-three people cannot work on one feature, and five people cannot carry an entire frontend.

Groups that moved: NEURALTEAM no category → BE · **DRAGONS** no category → FE · **ASTERIX** no category → FE · **GRP14** ML → FE · **GRP4** ML → UI/UX · **NEXAAI** ML → UI/UX · **GRP12** no category → SPD. Groups shown in **bold** above are the ones that changed. ML still has the most groups, on purpose — it carries face recognition, the LLM ask path, the agent loop and evals.

Which directory belongs to which track

PATH	TRACK	WHAT LIVES THERE
`backend/core/`	**BE**	Models (already written), serializers, views, URLs, admin, migrations
`backend/config/`	**BE**	Settings, DRF + JWT config, CORS, root URLconf
`frontend/src/app/`	**FE**	Screens, routing, auth context, route guards
`frontend/src/services/`	**FE**	API client, token refresh, error handling
`frontend/src/styles/`	**UI/UX**	Design tokens, theme, dark mode
`frontend/src/app/components/ui/`	**UI/UX**	Shared primitives and the six extracted components
`backend/faceid/`	**ML**	ONNX embeddings, cosine, verify, threshold sweep
`backend/llm/ agent/ evals/`	**ML**	Claude client, the ask endpoint, agent loop, eval set
`backend/core/queries.py`	**ML**	The bounded query functions the agent is allowed to call
`backend/presence/`	**SPD**	Campus IP, session codes, audit log, haversine
`backend/deploy/` `.github/workflows/`	**SPD**	Caddy, systemd, CI, deployment

Your track is where you start, not a fence. If your group finishes its backlog, the next thing you do is review another group's pull request — that is real work, and it counts.

BE Backend

4 groups · 18 students

Django, DRF, JWT, and the data model everyone else depends on.

> **Your mission.** You own the truth. Every other track reads what you write. If the API is wrong, the app is wrong no matter how good it looks.

GRP5 · 6 | GRP7 · 5 | CLAUDE · 4 | NEURALTEAM · 3

You own: `backend/core/ · backend/config/`

Sprint 1 — the backlog

DONE	TASK	WHAT IT MEANS
☐	**Replace the URLconf stub**	`backend/core/urls.py` answers `/api/health/` and nothing else, so a fresh clone starts. The seven real endpoints are listed in a comment at the top of it. **Everyone else is waiting on these.**
☐	**Wire up DRF and JWT**	`pip install djangorestframework-simplejwt django-cors-headers`, then add the `REST_FRAMEWORK`, `SIMPLE_JWT` and CORS blocks to `backend/config/settings.py` — there is no `REST_FRAMEWORK` block there at all.
☐	**Do not rewrite the models**	`backend/core/models.py` is finished: nine models, already carrying `photo`, lat/lng/accuracy, `face_score` and a `signals` JSON field. Read it, then run your first migration.
☐	**Serializers, views, admin**	`serializers.py`, `views.py`, `admin.py` in `backend/core/`. Register *every* model in admin — the reference registers one of ten, so you cannot inspect anything.
☐	**The eight endpoints**	`auth/login/` · `auth/refresh/` · `auth/me/` · `attendance/mark/` (multipart) · `attendance/today/` · `attendance/my-records/` · `admin/overview/?date=` · `health/`
☐	**Marking rules**	One mark per student per IST day → **409**. Block Sundays and `Holiday` rows. Wrap the write in `transaction.atomic()`, with a real `UniqueConstraint` behind it — a check-then-write is a race, not a constraint.
☐	**Seed the database**	A `seed_demo` management command with a **fixed random seed**, so every team's checkpoint number matches the one on the projector.

Who takes what

GROUP	FIRST PIECE OF WORK
GRP5	Auth: login, refresh, `/api/auth/me/`, permission classes
GRP7	The mark endpoint and every one of its rules
CLAUDE	Read endpoints, admin overview, Django admin, and the seed command
NEURALTEAM	`backend/core/urls.py` first, then settings, CORS and migrations

Four defects in the reference app. Fix them; do not copy them.

`pmutwd_attendance/be` is real production code, and its auth is theatre. **(1)** Plaintext password comparison — use `django.contrib.auth`. **(2)** Login returns `str(uuid.uuid4())`, never stored, never checked — and the frontend sends it as a Bearer token forever. **(3)** All 27 endpoints `AllowAny`; no view reads `request.user`. **(4)** `attendance_datetime` comes from the client, so any past date can be backfilled.

Start here. That project's stakeholder asked for `GET /api/auth/me`. Five lines of code — and **impossible to build there**, because the token meant nothing, so the server had no way to know who was asking. Auth is architecture, not a feature you add later.

Done looks like: `curl -s localhost:8000/api/health/` returns `{"ok":true}`, and a `POST /api/attendance/mark/` without a valid token returns **401**, not 201.

FE Frontend

4 groups · 19 students

React, the router, and the two browser APIs this whole app rests on.

> **Your mission.** You own what a student actually touches. A phone, one thumb, thirty seconds before class starts. If marking attendance takes longer than the paper sheet, we have built nothing.

DRAGONS · 5 | ASTERIX · 5 | GRP8 · 5 | GRP14 · 4

You own: `frontend/src/app/` · `frontend/src/services/`

Sprint 1 — the backlog

DONE	TASK	WHAT IT MEANS
☐	**Read what is already there**	`frontend/` is scaffolded and runs: Vite + React 18 + TS + Tailwind v4 + react-router v7, the `@` alias, and a dev proxy so `/api` reaches Django with no CORS. `npm install && npm run dev`, then delete a stub and start.
☐	**Keep the type-checker honest**	`npm run typecheck` runs `tsc -b --noEmit` and really does fail on a type error — plain `tsc --noEmit` checks nothing with this config. Run it before every pull request.
☐	**The API layer**	`services/api.ts`: fetch wrapper, `Authorization: Bearer`, and the **401 → refresh → retry** interceptor the reference never had.
☐	**Auth plumbing**	`context/AuthContext.tsx` is a stub whose `login()` throws and whose `isAuthenticated` is always false — which is why every guarded route currently bounces to `/login`. Make it real.
☐	**Four screens**	`Login` · `MarkAttendance` · `MyRecords` · `AdminDashboard`. Each is a placeholder today, naming its owner and its task list. Replace them.
☐	**Camera and geolocation**	Port `fe/src/app/components/employee/MarkAttendance.tsx:224–332` and **nothing else from that file** — it is 1,285 lines and about 110 of them matter.
☐	**The admin pie chart**	recharts donut, matching `ManagerDashboard.tsx:214–218`: present / leave / not-marked, with a live count beside each slice.

Who takes what

GROUP	FIRST PIECE OF WORK
GRP8	Scaffold, routing, `services/api.ts` and the refresh interceptor
DRAGONS	`MarkAttendance` — camera, canvas, capture, retake
GRP14	`Login` and `MyRecords`
ASTERIX	`AdminDashboard` and the recharts donut

> **Two patterns in that file were earned the hard way. Keep both.**
>
> **(1)** The stream is attached to the video element in a `useEffect` (`:75–80`), not inside `startCamera()` — because the `<video>` element has not mounted yet when the stream arrives. **(2)** Geolocation tries high accuracy, and on failure retries with `enableHighAccuracy:false` (`:319–331`). Also keep both artefacts of capture: a dataURL for the on-screen preview and a `File` for the upload.

You will hit this on Day 4, and it will look like your bug. `getUserMedia()` and the Geolocation API only work in a *secure context*. Over plain HTTP `navigator.mediaDevices` is not blocked — it is `undefined`, so you get a `TypeError` that reads exactly like an application error. `localhost` counts as secure. `http://<server-ip>` does not.

Done looks like: On a real phone, over HTTPS: camera opens, photo captures, coordinates appear, submit returns 201, and a second submit the same day returns 409 with a message a human can read.

UI / UX

UI/UX

4 groups · 15 students

The design system, and making the components actually use it.

> **Your mission.** You own how it feels and whether it holds together. Not decoration — a shared vocabulary that stops four frontend groups from inventing four different buttons.

GRP4 · 5 | MINIONS · 4 | AGENTS · 4 | NEXAAI · 2

You own: `frontend/src/styles/` · `frontend/src/app/components/ui/`

Sprint 1 — the backlog

DONE	TASK	WHAT IT MEANS
☐	**Design tokens**	`theme.css` using the Tailwind v4 `@theme inline` pattern from `pmutwd_attendance/fe/src/styles/theme.css`. Pick a Basar palette — do not just copy theirs.
☐	**Bind components to tokens**	Install only the shadcn primitives that get used, and make them read from the tokens. A token nobody references is a comment.
☐	**Build the six shared components**	`AppHeader` · `PhotoModal` · `MonthCalendar` + legend · `AttendanceRow` · `StatCard` · `EmptyState`. Every one of these is copy-pasted two or three times in the reference app.
☐	**Status colour semantics**	Decide once, write it down, and hand it to FE: present / absent / leave / holiday / today. Five states, five colours, one source of truth.
☐	**Mobile first**	This gets used standing up, on a phone, in a classroom, in five minutes. Design that, then let the desktop fall out of it.
☐	**Dark mode**	Via the `.dark` token block. It costs almost nothing if tokens are done properly — and it is the test of whether they are.

Who takes what

GROUP	FIRST PIECE OF WORK
GRP4	`MonthCalendar` + legend, `AttendanceRow`, `StatCard` — the three the reference copy-pasted most
MINIONS	Tokens, palette, typography scale, dark mode
AGENTS	`AppHeader`, `PhotoModal`, `EmptyState`
NEXAAI	Status colour semantics and the written handoff to FE — two people, so the scope is deliberately small

The cautionary tale is sitting in the reference repo.

It vendors **49** shadcn components. App code imports **6** — all from a single screen. The three main screens ignore the design system entirely and hardcode Tailwind utility strings, and each one re-implements its own `<header>` inline. There is a full 353-line themed chart wrapper that nothing imports, and a `--custom-blue` token no component references. **A design system that nothing uses is not a design system.** Your job is the binding, not the library.

Design the failure states too. No camera permission. No GPS fix. Offline. Already marked today. Marked outside class hours. Those five screens are most of what a real user will actually see, and they are the ones nobody remembers to design.

Done looks like: Change one token value in `theme.css` and watch the colour move on all four screens. If you have to edit a component to change a colour, you are not done.

ML & AI

4 groups · 16 students

Face embeddings, thresholds, the LLM ask path, and evals.

> **Your mission.** You own the parts that can be wrong in ways nobody notices. A threshold is a product decision wearing a number's clothes, and an agent that answers confidently and incorrectly is worse than no agent.

GRP11 · 5 | ANUSHA · 4 | BALAJI · 4 | FEMMEFORCE · 3

You own: `backend/ → faceid/ · llm/ · agent/ · evals/ · analysis/ · core/queries.py`

Sprint 1 — the backlog

DONE	TASK	WHAT IT MEANS
☐	**Embeddings**	`backend/faceid/embed.py` : ONNX detector + MobileFaceNet/ArcFace → 512-d vector. Three pure wheels, no C++ compiler. Then `cosine(a, b)` .
☐	**Verification**	`verify(roll_no, image_bytes) -> float` — best cosine similarity against that student's enrolled vectors. `EnrolmentPhoto.vector()` already exists in the model; use it.
☐	**Threshold sweep**	`backend/faceid/sweep.py` — sweep the threshold across enrolled photos, plot false-accept and false-reject. **Each group picks its own number and defends it out loud.**
☐	**Liveness, and its limits**	Two frames 800 ms apart; reject if byte-identical or cosine ≈ 1.000. Then hold a photo of a face up to the camera and watch it pass anyway.
☐	**Query functions**	`backend/core/queries.py` : `attendance_percentage()` (worked example), `students_below_threshold()` , `attendance_by_subject()` , and one more. `pytest tests/test_queries.py` → 4 passed.
☐	**The LLM ask**	`backend/llm/client.py` → class proxy, `claude-haiku-4-5-20251001` . `POST /api/ask/` returns schema-valid JSON. Then `backend/agent/loop.py` , hand-written.
☐	**Evals**	`backend/evals/run.py` — at least 20 cases, a before-score and an after-score, and one paragraph on what changed between them.

Who takes what

GROUP	FIRST PIECE OF WORK
ANUSHA	`embed.py` , `cosine()` , the ONNX pipeline
BALAJI	`verify()` , enrolment, then the threshold sweep and liveness — you cannot sweep a threshold until `verify()` returns a score, so it is one job
FEMMEFORCE	`backend/core/queries.py` and pandas analysis
GRP11	`backend/llm/client.py` , `/api/ask/` , agent loop, evals

Why not just send the photo to a vision API?

Two independent reasons. Anthropic's usage policy prohibits facial recognition. And decisively for this course: **a vision API cannot return a similarity score.** No score means no threshold, no false-accept / false-reject trade-off — and that trade-off is the entire lesson. So: local ONNX, on our own box.

The sentence we are aiming at: *"I chose 0.62 because a false reject costs a student their exam eligibility, and a false accept costs the institution very little."* Say that and you have stopped being someone who writes code. Also note: `backend/core/queries.py` is the **only** Python the LLM will ever be allowed to run. Everything an admin can ask is bounded by what you write there.

Done looks like: Your own face → 200. Your partner's face → 403. And you can state your threshold and both of its error rates without looking them up.

SPD

Platform

4 groups · 14 students

Presence signals, HTTPS, deployment, CI/CD, and the audit trail.

> **Your mission.** You own whether any of this can be believed. Attendance nobody can trust is a spreadsheet with extra steps — and an app that is down at 5pm is worth nothing at all.

BHUVANESHWARI · 5 **ROCKSTARS · 3** **MINIBYTES · 3** **GRP12 · 3**

You own: `backend/presence/ · backend/deploy/ · .github/workflows/`

Sprint 1 — the backlog

DONE	TASK	WHAT IT MEANS
☐	**Campus IP check**	`check_campus_ip(request)` against `CAMPUS_CIDR`. **Read `X-Forwarded-For`, not `REMOTE_ADDR`** — behind a reverse proxy, `REMOTE_ADDR` is the proxy.
☐	**Rotating projector code**	`verify_session_code(code, student)`. 60-second TTL, and `CodeRedemption` makes it single-use per student. You have to be looking at the screen.
☐	**haversine(), unused on purpose**	Write it. Test it. Log the distance. **Never gate on it.** Be ready to explain that to someone who thinks it is a bug.
☐	**Audit log**	Write an `AuditLog` row on every attempt — accepted or rejected — with the reason. The rejections are the interesting half.
☐	**HTTPS**	Caddy with automatic TLS, one subdomain per group. Without this the camera and GPS simply do not work, so this blocks two other tracks.
☐	**Keep it running**	A systemd unit, `.env` handling, and `verify.sh`. It has to come back up after `sudo reboot` without you touching it.
☐	**CI/CD**	GitHub Actions: lint + `pytest` + `vite build` on every PR into `dev`; deploy on `main`.

Who takes what

GROUP	FIRST PIECE OF WORK
BHUVANESHWARI	`backend/presence/signals.py` — IP check, session codes, haversine
MINIBYTES	Caddy, TLS, systemd, `.env`, `verify.sh`
ROCKSTARS	GitHub Actions, branch protection, and secret scanning
GRP12	The audit log — a row on every attempt, accepted or rejected, with the reason

Two gates and one log. Know which is which.

Campus IP — works indoors, campus-level only, and you must physically be on the network. **Gate on it. Projector code** — works indoors, room-level, and someone can WhatsApp it; the 60-second TTL and single use are what make that survivable. **Gate on it. GPS** — unreliable indoors, not room-level, and trivially faked from devtools in about four seconds. **Log it and never gate on it.** Spoofed coordinates should still return 200, and you should be able to say why.

The reference deploy pipeline is the anti-pattern. `deploy-backend.yml` is `git pull && migrate && restart gunicorn` over SSH, against a live box with the SQLite file sitting in the working tree. No tests, no backup, no rollback. Improve on it, and write down why in the PR description. Also: `git log -p --all | grep -c "sk-ant"` must return **0**.

Done looks like: The projector code returns 200. The same code 90 seconds later returns 403. Spoofed coordinates still return 200 — **and you can explain that to the room.** Then `sudo reboot`, and 60 seconds later the URL is up.

Teams and Branches

Nineteen groups · 106 students. Find your group, then your track page.

GROUP	TRACK	YOUR BRANCH	MEMBERS
GRP5	BE	`team/grp5`	6
GRP7	BE	`team/grp7`	5
CLAUDE	BE	`team/claude`	4
NEURALTEAM	BE	`team/neuralteam`	3
DRAGONS	FE	`team/dragons`	5
ASTERIX	FE	`team/asterix`	5
GRP8	FE	`team/grp8`	5
GRP14	FE	`team/grp14`	4
GRP4	UI/UX	`team/grp4`	5
MINIONS	UI/UX	`team/minions`	4
AGENTS	UI/UX	`team/agents`	4
NEXAAI	UI/UX	`team/nexaai`	2
GRP11	ML	`team/grp11`	5
ANUSHA	ML	`team/anusha`	4
BALAJI	ML	`team/balaji`	4
FEMMEFORCE	ML	`team/femmeforce`	3
BHUVANESHWARI	SPD	`team/bhuvaneshwari`	5
ROCKSTARS	SPD	`team/rockstars`	3
MINIBYTES	SPD	`team/minibytes`	3
GRP12	SPD	`team/grp12`	3

Why there are no names in this document

This copy lives in a **public** repository, which search engines index and which is effectively permanent. Student names and group email addresses do not belong in it. The printed copy handed out in class carries the full roster; this one carries what you need to do the work and nothing else. **Apply the same rule to everything you commit** — attendance photos, face embeddings and the database are all in `.gitignore` for exactly this reason. Check before you `git add .`.

Not sure which group you are in? Ask in class — the full roster is on the printed brief.